CHAPTER 7

# Decorator Pattern

This chapter covers the Decorator pattern.

## GoF Definition

Attach additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality.

## Concept

This pattern promotes the concept that your class should be closed for modification but open for extension. In other words, you can add a functionality without disturbing the existing functionalities. The concept is useful when you want to add some special functionality to a specific object instead of the whole class. This pattern prefers object composition over inheritance. Once you master this technique, you can add new responsibilities to an object without affecting the underlying classes.

## Real-Life Example

Suppose you own a single-story house and you decide to build a second floor on top of it. Obviously, you may not want to change the architecture of the ground floor. But you may want to change the design of the architecture for the newly added floor without affecting the existing architecture.

Figure 7-1, Figure 7-2, and Figure 7-3 illustrate this concept.



© Vaskaran Sarcar 2018
V. Sarcar, *Design Patterns in C#*, https://doi.org/10.1007/978-1-4842-3640-6_7

www.EBooksWorld.ir




***Figure 7-1.*** *Original house*




***Figure 7-2.*** *Original house with a decorator (the additional structure is built on top of original structure)*

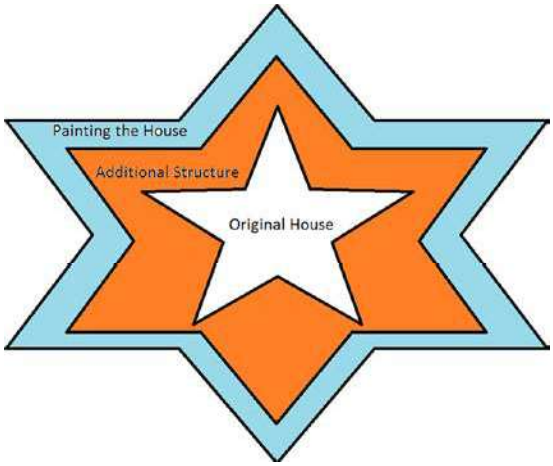



***Figure 7-3.*** *Creating an additional decorator from an existing decorator and modifying the house (now painting the house)*



www.EBooksWorld.ir

**Note** The case shown in Figure 7-3 is optional. You can use an existing decorator object to enhance the behavior in this way, or you can create a new decorator object and put all the new behavior in it.

## Computer World Example

Suppose in a GUI-based toolkit you want to add some border properties. You could do this with inheritance, but that cannot be treated as an ultimate solution because you do not have absolute control over everything from the beginning. So, this technique is static in nature.

Decorators offer a flexible approach. They promote the concept of dynamic choices. For example, you can surround the component in another object. The enclosing object is termed a *decorator*, and it must conform to the interface of the component that it decorates. It will forward the requests to the component, and it can perform additional operations before or after those requests. In fact, you can add an unlimited number of responsibilities with this concept.

**Note** The I/O streams implementations in both .NET Framework and Java use the concept of Decorator pattern.

## Illustration

In this example, I have not modified the core `MakeHouse()` method. I have created two additional decorators, `ConcreteDecoratorEx1` and `ConcreteDecoratorEx2`, to serve my needs, but I have kept the original structure intact.



www.EBooksWorld.ir

## Class Diagram

Figure 7-4 shows the class diagram.

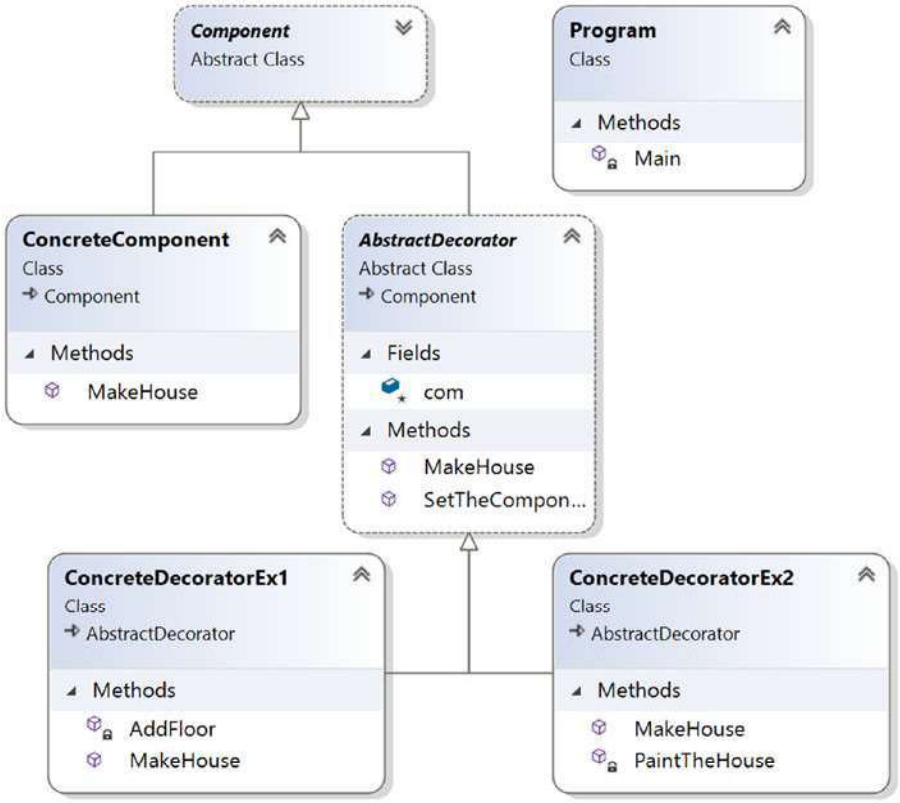



***Figure 7-4.*** *Class diagram*



www.EBooksWorld.ir

## Solution Explorer View

Figure 7-5 shows the high-level structure of the parts of the program.

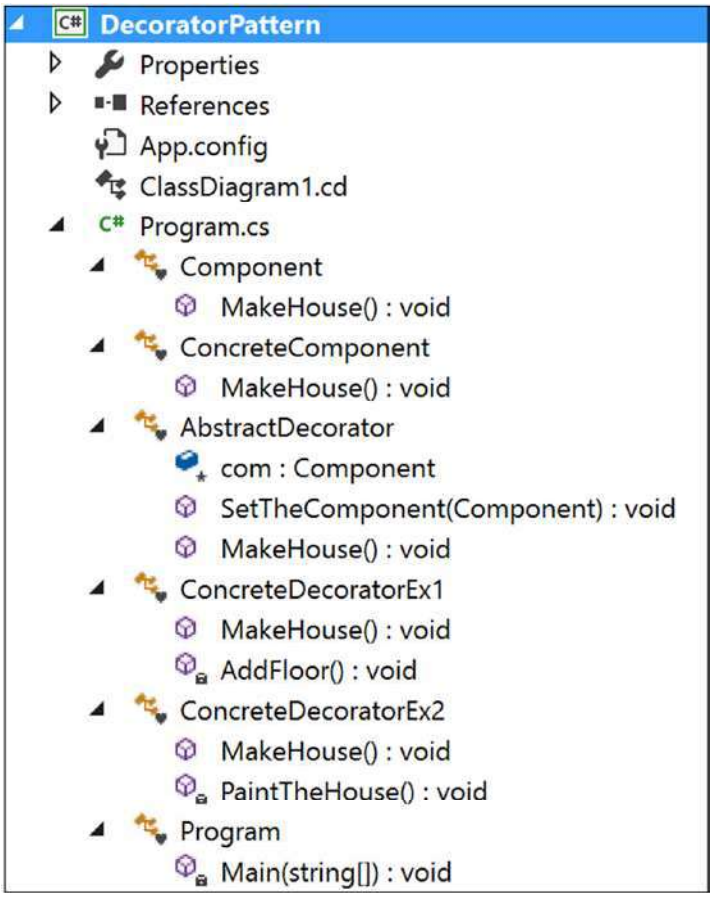



***Figure 7-5.*** *Solution Explorer View*



www.EBooksWorld.ir

## Implementation

Here's the implementation:

```
using System;

namespace DecoratorPattern
{
    abstract class Component
    {
        public abstract void MakeHouse();

    }
    class ConcreteComponent : Component
    {
        public override void MakeHouse()
        {
            Console.WriteLine("Original House is complete. It is closed for
            modification.");
        }
    }

    abstract class AbstractDecorator : Component
    {
        protected Component com ;
        public void SetTheComponent(Component c)
        {
            com = c;
        }
        public override void MakeHouse()
        {
            if (com != null)
            {
                com.MakeHouse();//Delegating the task
            }
        }

    }
```



www.EBooksWorld.ir

```
class ConcreteDecoratorEx1 : AbstractDecorator
{
    public  override void MakeHouse()
    {
        base.MakeHouse();
        Console.WriteLine("***Using a decorator***");
        //Decorating now.
        AddFloor();
        //You can put additional stuff as per your needs.
    }
    private void AddFloor()
    {
        Console.WriteLine("I am making an additional floor on top of
        it.");
    }
}
class ConcreteDecoratorEx2 : AbstractDecorator
{
    public  override void MakeHouse()
    {
        Console.WriteLine("");
        base.MakeHouse();
        Console.WriteLine("***Using another decorator***");
        //Decorating now.
        PaintTheHouse();
        //You can add additional stuffs as per your need
    }
    private void PaintTheHouse()
    {
        Console.WriteLine("Now I am painting the house.");
    }
}
```



www.EBooksWorld.ir

```
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("***Decorator pattern Demo***\n");
            ConcreteComponent cc = new ConcreteComponent();

            ConcreteDecoratorEx1 decorator1 = new ConcreteDecoratorEx1();
            decorator1.SetTheComponent(cc);
            decorator1.MakeHouse();

            ConcreteDecoratorEx2 decorator2 = new ConcreteDecoratorEx2();
            //Adding results from decorator1
            decorator2.SetTheComponent(decorator1);
            decorator2.MakeHouse();

            Console.ReadKey();
        }
    }
}
```

## Output

Here's the output. Notice the lines in bold.

```
***Decorator pattern Demo***

Original House is complete. It is closed for modification.
***Using a decorator***
I am making an additional floor on top of it.

Original House is complete. It is closed for modification.
***Using a decorator***
I am making an additional floor on top of it.
***Using another decorator***
Now I am painting the house.
```



www.EBooksWorld.ir

# Q&A Session

1. **Can you explain how composition is promoting dynamic behavior that inheritance cannot?**

   Answer:

   When a derived class inherits from a base class, it actually inherits the behavior of the base class at that time only. Though different subclasses can extend the base/parent class in different ways, this type of binding is known at compile time. So, the method is static. But by using the concept of composition, as in the previous example, you get dynamic behavior.

   When you design a base/parent class, you may not have enough visibility about what kind of additional responsibilities your clients may want in some later phase. Since the constraint is that you cannot modify the existing code, in this case object composition not only outclasses inheritance, but it also ensures that you are not introducing bugs in the old architecture.

   Lastly, in this context, you must try to remember one key design principle that says *classes should be open for extension but closed for modification*.

2. **What are the key advantages of using a decorator?**

   Answer:

   - The existing structure is untouched, so you cannot introduce bugs there.
   - New functionalities can be added to an existing object easily.
   - You do not need to predict/implement all the supported functionalities at once (in other words, in the initial design phase). You can develop incrementally. For example, you can add decorator objects one by one to support your needs. You must acknowledge that if you make a complex class first and then want to extend the functionalities, that will be a tedious process.



www.EBooksWorld.ir

3. **How is the overall design pattern different from inheritance?**

   Answer:

   You can add or remove responsibilities by simply attaching or detaching decorators. But with simple inheritance techniques, you need to create new classes for new responsibilities. So, it is possible that you will end up with a complex system.

   Consider the example again. Suppose you want to add a new floor, paint the house, and do some extra work. To fulfill this need, you can start with `decorator2` because it is already providing the support to add a floor and paint. Then you need to add a simple wrapper to complete those additional responsibilities.

   But if you start with inheritance from the beginning, then you may have multiple subclasses, in other words, one for adding a floor and one for painting the house, as shown in Figure 7-6 (a hierarchical inheritance).

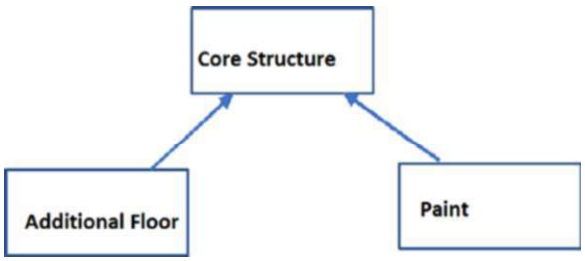



***Figure 7-6.*** *A hierarchical inheritance*

So, if you need an additional painted floor with some extra features, you may need to end up with a design like in Figure 7-7.



www.EBooksWorld.ir

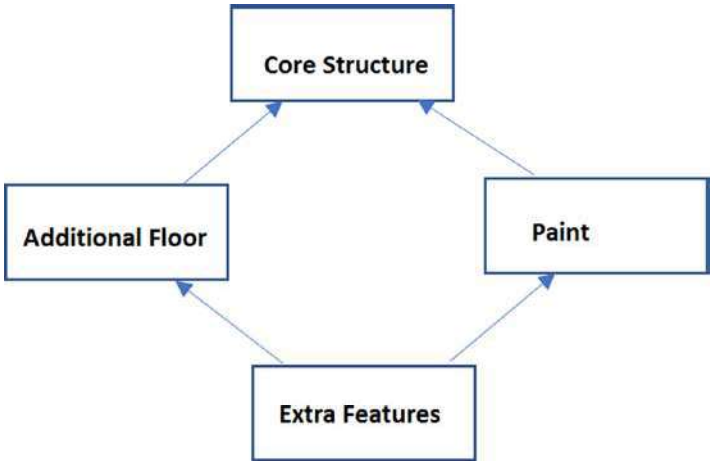



***Figure 7-7.*** *A class (Extra Features) needs to inherit from multiple base classes*

Now you will feel the heat of the "diamond effect" because in many programming languages, including C#, multiple base classes are not allowed.

You will also discover that the inheritance mechanism is not only much more challenging and time-consuming compared to the Decorator pattern, but it may promote duplicate code in your application. Lastly, do not forget that inheritance promotes only compile-time binding (not dynamic binding).

4. **Why are you creating a class with a single responsibility? You could make a subclass that can simply add a floor and then paint. In that case, you may end up with fewer subclasses. Is this understanding correct?**

   Answer:

   If you are familiar with SOLID principles, you will know that there is a principle called *single responsibility*. The idea behind this principle is that each class should have a responsibility over a single part of the functionality provided in the software. The Decorator pattern is effective when you use the single responsibility principle because you can simply add or remove responsibilities dynamically.



www.EBooksWorld.ir

5. **What are the disadvantages associated with this pattern?**

   Answer:

   I believe that if you are careful, there are no significant disadvantages. But if you create too many decorators in the system, it will be hard to maintain and debug. So, in that case, they can create unnecessary confusion.

6. **In the example, the AbstractDecorator class is abstract, but there is no abstract method in it. How is this possible?**

   Answer:

   In C#, a class can be abstract without containing any abstract method, but the reverse is not true. In other words, if a class contains at least one abstract method, it means that the class itself is incomplete and you are forced to mark it with the `abstract` keyword.

   Also, if you read the comment in Figure 7-8, you'll understand that you are delegating the task to a concrete decorator in this case because you want to use and instantiate the concrete decorators only.

```
abstract class AbstractDecorator : Component
{
    protected Component com ;
    public void SetTheComponent(Component c)
    {
        com = c;
    }
    public override void MakeHouse()
    {
        if (com != null)
        {
            com.MakeHouse();//Delegating the task
        }
    }

}
```

***Figure 7-8.*** *An abstract class-AbstractDecorator*



www.EBooksWorld.ir

So, in this example, you cannot simply instantiate an `AbstractDecorator` instance because it is marked with the `abstract` keyword.

The following line will create a compilation error (Figure 7-9):

```
AbstractDecorator abstractDecorator = new AbstractDecorator();
```




***Figure 7-9.*** *Error message*

7. **In this example, instead of using concrete decorators, you could use the concept of polymorphism to generate the same output.**

```
AbstractDecorator decorator1 = new ConcreteDecoratorEx1();
decorator1.SetTheComponent(cc);
decorator1.MakeHouse();

//ConcreteDecoratorEx2 decorator2 = new ConcreteDecoratorEx2();
AbstractDecorator decorator2 = new ConcreteDecoratorEx2();
//Adding results from decorator1
decorator2.SetTheComponent(decorator1);
decorator2.MakeHouse();
```

**Is this understanding correct?**

Answer:

Yes.

8. **Then using polymorphism, can you reduce the lines of code in this way?**

```
AbstractDecorator decorator1 = new ConcreteDecoratorEx1();
decorator1.SetTheComponent(cc);
decorator1.MakeHouse();
/Adding results from decorator1
decorator1.SetTheComponent(decorator1);
decorator2.MakeHouse();
```



www.EBooksWorld.ir

Answer:

No. First, is this easily readable? Second, you will receive a `StackOverflowException` at runtime.

9. **Is it mandatory to use decorators for dynamic binding only?**

   Answer:

   No. You can use the concept for both static and dynamic binding. But dynamic binding is its strength, so I have concentrated on that here. The GoF definition also focuses on dynamic binding only.



www.EBooksWorld.ir